



01 PAO25-25 - PYTHON, Data Types



Luis Pilaguano

Caso práctico: Árboles de decisión

En este caso de uso práctico se pretende resolver un problema de detección de malware en dispositivos Android mediante el análisis del tráfico de red que genera el dispositivo mediante el uso de árboles de decisión.

Conjunto de datos: Detección de malware en Android

Descripción

El sofisticado y avanzado malware de Android puede identificar la presencia del emulador utilizado por el analista de malware y, en respuesta, modificar su comportamiento para evitar ser detectado. Para solucionar este problema, instalamos las aplicaciones de Android en el dispositivo real y capturamos su tráfico de red. Consulte nuestro Sandbox de Android disponible públicamente.

El conjunto de datos de CICAAGM se obtiene instalando las aplicaciones de Android en los smartphones reales de forma semiautomatizada. El conjunto de datos se genera a partir de 1900 aplicaciones pertenecientes a las tres categorías siguientes:

1. Adware (250 aplicaciones)

Airpush: Diseñado para enviar anuncios no solicitados a los sistemas del usuario con el fin de robar información.

Dowgin: Diseñado como una biblioteca de publicidad que también puede robar la información del usuario.

Kemoge: Diseñado para controlar el dispositivo Android del usuario. Este adware es un híbrido de botnet y se disfraza de aplicaciones populares mediante reempaquetado.

Mobidash: Diseñado para mostrar anuncios y comprometer la información personal del usuario.

Shuanet: similar a Kemoge, Shuanet también está diseñado para apoderarse del dispositivo de un usuario.

2. Malware general (150 aplicaciones)

AVpass: Diseñado para distribuirse bajo la apariencia de una aplicación de reloj.

FakeAV: diseñado como una estafa que engaña al usuario para que compre una versión completa del software para remediar infecciones inexistentes.

FakeFlash/FakePlayer: Diseñado como una aplicación Flash falsa para dirigir a los usuarios a un sitio web (después de una instalación exitosa).

GGtracker: Diseñado para el fraude de SMS (envía mensajes SMS a un número de tarifa premium) y el robo de información.

Penetho: Diseñado como un servicio falso (una herramienta de hackeo para dispositivos Android que permite descifrar la contraseña de la red wifi). El malware también puede infectar el ordenador del usuario mediante archivos adjuntos de correo electrónico infectados, actualizaciones falsas, medios externos y documentos infectados.

3. Benigno (1500 aplicaciones)

Mercado GooglePlay 2015 (top gratuitos populares y top gratuitos nuevos).

Mercado GooglePlay 2016 (top gratuitos populares y top gratuitos nuevos).

Ficheros de datos

Archivos .pcap: el tráfico de red tanto del malware como del benigno (20% malware y 80% benigno).

Archivos .csv: la lista de características de tráfico de red extraídas generadas por el medidor de flujo CIC

Descarga de los ficheros de datos

()[<https://www.unb.ca/cic/datasets/android-adware.html>]

Referencias adicionales sobre el conjunto de datos

Arash Habibi Lashkari, Andi Fitriah A. Kadir, Hugo Gonzalez, Kenneth Fon Mbah y Ali A. Ghorbani, "Hacia un marco basado en red para la detección y caracterización de malware en Android", en las actas de la 15.^a Conferencia Internacional sobre Privacidad, Seguridad y Confianza, PST, Calgary, Canadá, 2017.

Importaciones

En [4]:

```
# Instalamos graphviz
!pip install graphviz
```

Acceso denegado.

Requirement already satisfied: graphviz in c:\users\santiago\anaconda3\envs \machine learning\lib\site-packages (0.20.1)

In [5]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import confusion_matrix, recall_score, f1_score, precision_
from pandas import DataFrame
```

In [6]:

```
import warnings
# Ignoramos algunos warnings que se producen por invocar el modelo sin el nombre
warnings.filterwarnings('ignore', category=UserWarning, message='.*X does not ha
```

Funciones auxiliares

In [7]:

```
# Construcción de una función que realice el particionado completo
def train_val_test_split(df, rstate=42, shuffle=True, stratify=None):
    strat = df[stratify] if stratify else None
    train_set, test_set = train_test_split(
        df, test_size=0.4, random_state=rstate, shuffle=shuffle, stratify=strat)
    strat = test_set[stratify] if stratify else None
    val_set, test_set = train_test_split(
        test_set, test_size=0.5, random_state=rstate, shuffle=shuffle, stratify=
    return (train_set, val_set, test_set)
```

In [8]:

```
def remove_labels(df, label_name):
    X = df.drop(label_name, axis=1)
    y = df[label_name].copy()
    return (X, y)
```

In [9]:

```
def evaluate_result(y_pred, y, y_prep_pred, y_prep, metric):
    print(metric.__name__, "WITHOUT preparation:", metric(y_pred, y, average='we
    print(metric.__name__, "WITH preparation:", metric(y_prep_pred, y_prep, aver
```

1. Lectura del conjunto de datos

```
In [11]: df = pd.read_csv('TotalFeatures-ISCXFlowMeter.csv')
```

2. Visualización del conjunto de datos

```
In [12]: df.head(10)
```

```
Out[12]:
```

	duration	total_fpackets	total_bpackets	total_fpkttl	total_bpkttl	min_fpkttl	min_bpkttl
0	1020586	668	1641	35692	2276876	52	52
1	80794	1	1	75	124	75	124
2	998	3	0	187	0	52	-1
3	189868	9	9	1448	6200	52	52
4	110577	4	6	528	1422	52	52
5	261876	7	6	1618	882	52	52
6	14	2	0	104	0	52	-1
7	29675	1	1	71	213	71	213
8	806635	4	0	239	0	52	-1
9	56620	3	2	1074	719	52	52

10 rows × 80 columns



```
In [13]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 631955 entries, 0 to 631954
```

```
Data columns (total 80 columns):
```

#	Column	Non-Null Count	Dtype
0	duration	631955 non-null	int64
1	total_fpackets	631955 non-null	int64
2	total_bpackets	631955 non-null	int64
3	total_fpktl	631955 non-null	int64
4	total_bpktl	631955 non-null	int64
5	min_fpktl	631955 non-null	int64
6	min_bpktl	631955 non-null	int64
7	max_fpktl	631955 non-null	int64
8	max_bpktl	631955 non-null	int64
9	mean_fpktl	631955 non-null	float64
10	mean_bpktl	631955 non-null	float64
11	std_fpktl	631955 non-null	float64
12	std_bpktl	631955 non-null	float64
13	total_fiat	631955 non-null	int64
14	total_biat	631955 non-null	int64
15	min_fiat	631955 non-null	int64
16	min_biat	631955 non-null	int64
17	max_fiat	631955 non-null	int64
18	max_biat	631955 non-null	int64
19	mean_fiat	631955 non-null	float64
20	mean_biat	631955 non-null	float64
21	std_fiat	631955 non-null	float64
22	std_biat	631955 non-null	float64
23	fpsh_cnt	631955 non-null	int64
24	bpsh_cnt	631955 non-null	int64
25	furg_cnt	631955 non-null	int64
26	burg_cnt	631955 non-null	int64
27	total_fhlen	631955 non-null	int64
28	total_bhlen	631955 non-null	int64
29	fPktsPerSecond	631955 non-null	float64
30	bPktsPerSecond	631955 non-null	float64
31	flowPktsPerSecond	631955 non-null	float64
32	flowBytesPerSecond	631955 non-null	float64
33	min_flowpktl	631955 non-null	int64
34	max_flowpktl	631955 non-null	int64
35	mean_flowpktl	631955 non-null	float64
36	std_flowpktl	631955 non-null	float64
37	min_flowiat	631955 non-null	int64
38	max_flowiat	631955 non-null	int64
39	mean_flowiat	631955 non-null	float64
40	std_flowiat	631955 non-null	float64
41	flow_fin	631955 non-null	int64
42	flow_syn	631955 non-null	int64
43	flow_rst	631955 non-null	int64
44	flow_psh	631955 non-null	int64
45	flow_ack	631955 non-null	int64
46	flow_urg	631955 non-null	int64
47	flow_cwr	631955 non-null	int64
48	flow_ece	631955 non-null	int64
49	downUpRatio	631955 non-null	float64
50	avgPacketSize	631955 non-null	float64
51	fAvgSegmentSize	631955 non-null	float64
52	fHeaderBytes	631955 non-null	int64
53	fAvgBytesPerBulk	631955 non-null	int64
54	fAvgPacketsPerBulk	631955 non-null	int64

```

55 fAvgBulkRate          631955 non-null int64
56 bVarianceDataBytes    631955 non-null float64
57 bAvgSegmentSize       631955 non-null int64
58 bAvgBytesPerBulk      631955 non-null int64
59 bAvgPacketsPerBulk    631955 non-null int64
60 bAvgBulkRate          631955 non-null int64
61 sflow_fpacket         631955 non-null int64
62 sflow_fbytes          631955 non-null int64
63 sflow_bpacket         631955 non-null int64
64 sflow_bbytes          631955 non-null int64
65 min_active            631955 non-null int64
66 mean_active           631955 non-null float64
67 max_active            631955 non-null int64
68 std_active            631955 non-null float64
69 min_idle              631955 non-null int64
70 mean_idle             631955 non-null float64
71 max_idle              631955 non-null int64
72 std_idle              631955 non-null float64
73 FFNEPD                631955 non-null int64
74 Init_Win_bytes_forward 631955 non-null int64
75 Init_Win_bytes_backward 631955 non-null int64
76 RRT_samples_clnt      631955 non-null int64
77 Act_data_pkt_forward  631955 non-null int64
78 min_seg_size_forward  631955 non-null int64
79 calss                  631955 non-null object
dtypes: float64(24), int64(55), object(1)
memory usage: 385.7+ MB

```

```
In [14]: df['calss'].value_counts()
```

```

Out[14]: calss
benign          471597
asware          155613
GeneralMalware   4745
Name: count, dtype: int64

```

Buscando correlaciones

```

In [15]: # Copiamos el conjunto de datos y transformamos la variable de salida a numérica
X = df.copy()
X['calss'] = X['calss'].factorize()[0]

```

```

In [16]: # Calculamos correlaciones
corr_matrix = X.corr()
corr_matrix["calss"].sort_values(ascending=False)

```

```

Out[16]: calss          1.000000
flow_fin          0.286175
min_seg_size_forward 0.258352
Init_Win_bytes_forward 0.129425
std_fpctl         0.123758
...
furg_cnt          NaN
burg_cnt          NaN
flow_urg          NaN
flow_cwr          NaN
flow_ece          NaN
Name: calss, Length: 80, dtype: float64

```

In [17]: `X.corr()`

Out[17]:

	duration	total_fpackets	total_bpackets	total_fpktl	total_bpktl
duration	1.000000	0.004837	0.004011	0.001673	0.003518
total_fpackets	0.004837	1.000000	0.924622	0.425756	0.904007
total_bpackets	0.004011	0.924622	1.000000	0.156780	0.997268
total_fpktl	0.001673	0.425756	0.156780	1.000000	0.090082
total_bpktl	0.003518	0.904007	0.997268	0.090082	1.000000
...	...	...	...	...	...
Init_Win_bytes_backward	0.029712	0.059224	0.058435	0.015991	0.053134
RRT_samples_clnt	0.003785	0.902713	0.997580	0.088422	0.999616
Act_data_pkt_forward	0.004838	0.999866	0.924746	0.425789	0.904129
min_seg_size_forward	0.082955	0.018198	0.015124	0.005477	0.012139
calss	0.067066	0.018377	0.019430	0.000679	0.019836

80 rows × 80 columns


In [18]: `# Se puede llegar a valorar quedarnos con aquellas que tienen mayor correlación
corr_matrix[corr_matrix["calss"] > 0.05]`

Out[18]:

	duration	total_fpackets	total_bpackets	total_fpktl	total_bpk
duration	1.000000	0.004837	0.004011	0.001673	0.00351
max_bpktl	0.042925	0.086255	0.086886	0.022088	0.07990
mean_bpktl	0.025117	0.139142	0.151761	0.018954	0.14643
std_fpktl	0.039350	0.010172	0.002331	0.011416	-0.00316
std_bpktl	0.048743	0.020324	0.014005	0.007763	0.00776
total_fiat	0.943898	0.002190	0.001718	0.000708	0.00154
min_fiat	0.841692	-0.001975	-0.002172	-0.000746	-0.00182
max_fiat	0.943438	-0.000420	-0.000714	-0.000389	-0.00054
mean_fiat	0.918036	-0.001970	-0.002241	-0.000728	-0.00190
std_flowpktl	0.036942	0.087741	0.087307	0.024298	0.08066
min_flowiat	0.841666	-0.001972	-0.002169	-0.000744	-0.00181
max_flowiat	0.999457	0.002273	0.001623	0.000611	0.00145
mean_flowiat	0.949299	-0.002050	-0.002292	-0.000748	-0.00197
flow_fin	0.068154	0.005461	0.004894	-0.001464	0.00511
flow_syn	0.020298	0.045730	0.043305	0.012057	0.03925
flow_rst	-0.003951	-0.004097	-0.004343	-0.001550	-0.00373
bVarianceDataBytes	0.042755	0.012731	0.006942	0.004947	0.00203
bAvgSegmentSize	0.025105	0.139151	0.151778	0.018949	0.14645
min_active	0.997972	0.002195	0.001495	0.000434	0.00141
mean_active	0.998911	0.002580	0.001852	0.000650	0.00172
max_active	0.999465	0.003261	0.002592	0.000919	0.00240
min_idle	0.997952	0.001258	0.000610	0.000113	0.00054
mean_idle	0.998901	0.001614	0.000922	0.000335	0.00081
max_idle	0.999458	0.002267	0.001617	0.000609	0.00145
Init_Win_bytes_forward	0.027610	0.050201	0.048190	0.013283	0.04357
Init_Win_bytes_backward	0.029712	0.059224	0.058435	0.015991	0.05313
min_seg_size_forward	0.082955	0.018198	0.015124	0.005477	0.01213
calss	0.067066	0.018377	0.019430	0.000679	0.01983

28 rows × 80 columns



3. División del conjunto de datos


```
In [19]: # Dividimos el conjunto de datos
train_set, val_set, test_set = train_val_test_split(X)
```

```
In [20]: X_train, y_train = remove_labels(train_set, 'calss')
X_val, y_val = remove_labels(val_set, 'calss')
X_test, y_test = remove_labels(test_set, 'calss')
```

4. Escalando el conjunto de datos

Es importante comprender que los árboles de decisión son algoritmos que no requieren demasiada preparación de los datos. Concretamente, no requieren la realización de escalado o normalización.

En este ejercicio se va a realizar escalado al conjunto de datos y se van a comparar los resultados con el conjunto de datos sin escalar. De esta forma se demuestra cómo aplicar preprocesamientos como el escalado puede incluso llegar a afectar al rendimiento del modelo.

```
In [21]: scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_val_scaled = scaler.transform(X_val)
```

5. Árbol de decisión

```
In [22]: from sklearn.tree import DecisionTreeClassifier
```

```
In [23]: MAX_DEPTH = 20

# Modelo entrenado con el conjunto de datos sin escalar
clf_tree = DecisionTreeClassifier(max_depth=MAX_DEPTH, random_state=42)
clf_tree.fit(X_train, y_train)
```

```
Out[23]: ▼ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=20, random_state=42)
```

```
In [24]: # Modelo entrenado con el conjunto de datos escalado
clf_tree_scaled = DecisionTreeClassifier(max_depth=MAX_DEPTH, random_state=42)
clf_tree_scaled.fit(X_train_scaled, y_train)
```

```
Out[24]: ▼ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=20, random_state=42)
```

Comenzar prediciendo con el propio conjunto de datos con el que se ha entrenado el algoritmo (train set), suele ser interesante para comprobar si se está produciendo overfitting.

```
In [25]: # Predecimos con el conjunto de datos de entrenamiento
y_train_pred = clf_tree.predict(X_train)
y_train_prep_pred = clf_tree_scaled.predict(X_train_scaled)
```

```
In [27]: # Comparamos resultados entre escalado y sin escalar
evaluate_result(y_train_pred, y_train, y_train_prep_pred, y_train, f1_score)
```

f1_score WITHOUT preparation: 0.95939228628057

f1_score WITH preparation: 0.9593952234926661

```
In [28]: # Predecimos con el conjunto de datos de validación
y_pred = clf_tree.predict(X_val)
y_prep_pred = clf_tree_scaled.predict(X_val_scaled)
```

```
In [29]: # Comparamos resultados entre escalado y sin escalar
evaluate_result(y_pred, y_val, y_prep_pred, y_val, f1_score)
```

f1_score WITHOUT preparation: 0.9336077973436508

f1_score WITH preparation: 0.93373075527604

Aunque los resultados son muy similares, el modelo sin escalar es más rápido y proporciona resultados ligeramente superiores. A partir de este punto, y, en función de los resultados que hemos visto, nos quedamos con la opción de no escalar nuestros conjuntos de datos.

6. Visualizando el limite de decision

```
In [31]: # Reducimos el número de atributos del conjunto de datos para visualizarlo mejor
X_train_reduced = X_train[['min_flowpkt1', 'flow_fin']]
```

```
In [32]: # Generamos un modelo con el conjunto de datos reducido
clf_tree_reduced = DecisionTreeClassifier(max_depth=2, random_state=42)
clf_tree_reduced.fit(X_train_reduced, y_train)
```

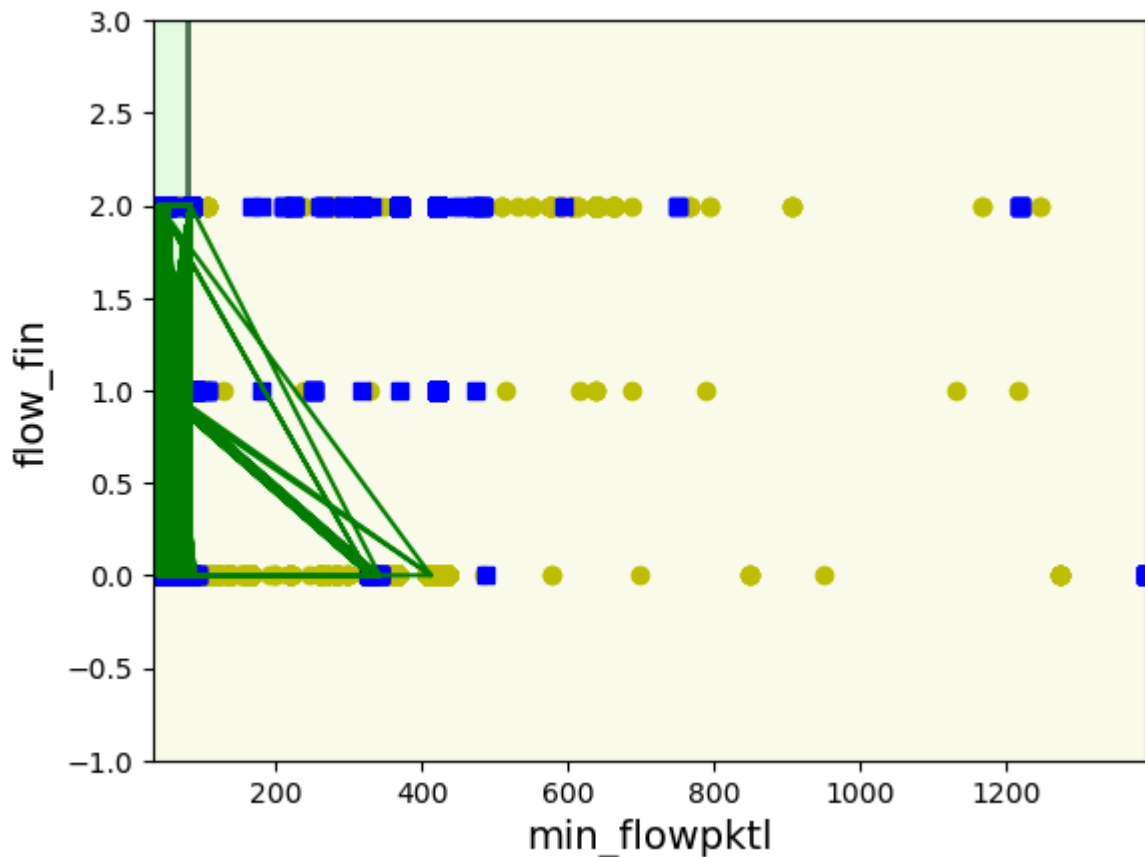
```
Out[32]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2, random_state=42)
```

```
In [33]: # Representamos gráficamente el límite de decisión construido
from matplotlib.colors import ListedColormap
import matplotlib.pyplot as plt
%matplotlib inline

def plot_decision_boundary(clf, X, y, plot_training=True, resolution=1000):
    mins = X.min(axis=0) - 1
    maxs = X.max(axis=0) + 1
    x1, x2 = np.meshgrid(
        np.linspace(mins[0], maxs[0], resolution),
        np.linspace(mins[1], maxs[1], resolution)
    )
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)
    custom_cmap = ListedColormap(['#fafabe', '#9898ff', '#a0faa0'])
    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap=custom_cmap)
    custom_cmap2 = ListedColormap(['#7d7d58', '#4c4c7f', '#507d50'])
    plt.contour(x1, x2, y_pred, cmap=custom_cmap2, alpha=0.8)
```

```
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo", label=" normal" )
plt.plot ( X[:, 0][ y == 1 ], X[:, 1][ y == 1 ], "bs" , etiqueta = "adwa
```

```
límite_de_decisión_del_gráfico ( árbol_clf_reducido , tren_X_reducido . valores
plt . mostrar ()
```



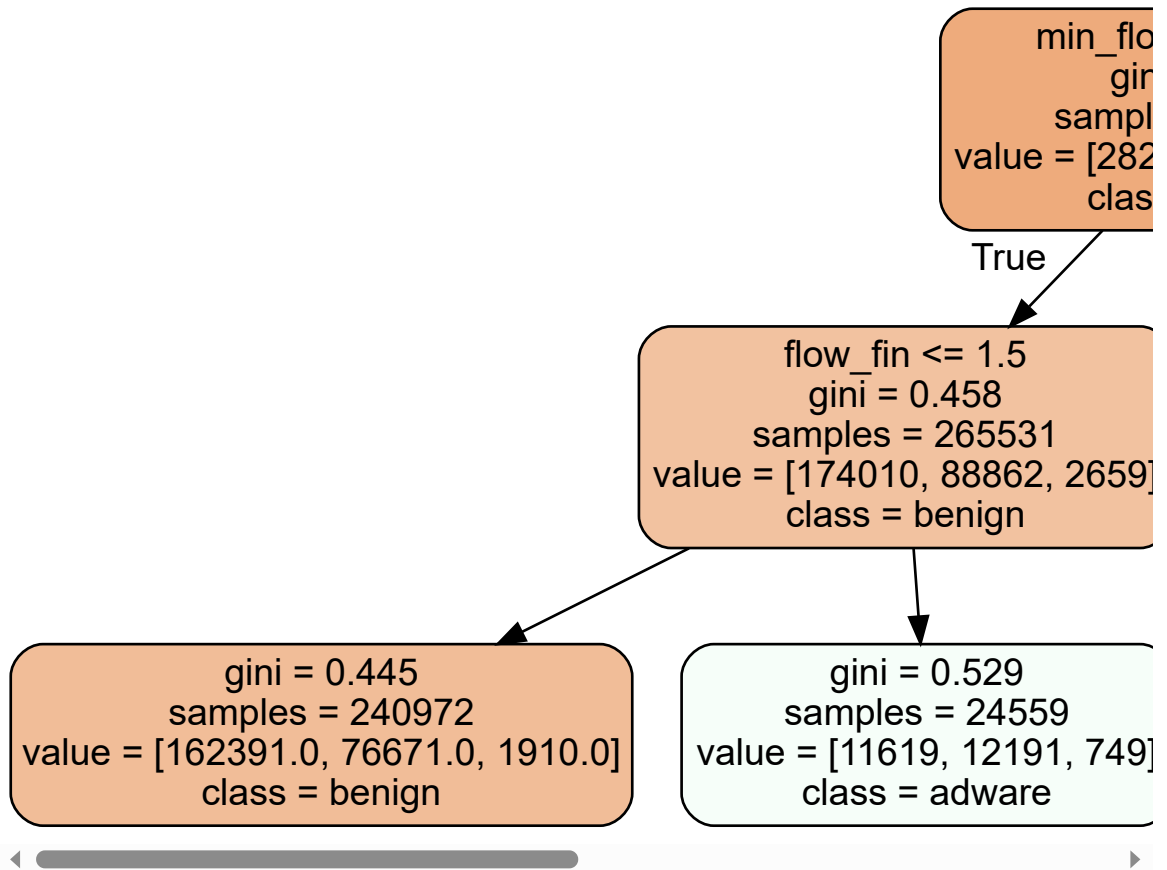
<Tamaño de la figura 1200x600 con 0 ejes>

En [35]: *# Pintamos el árbol para compararlo con la representación gráfica anterior* de sklearn.tree import export_graphviz from graphviz import Sou

```
export_graphviz (
    clf_tree_reduced ,
    out_file = "android_malware.dot" ,
    feature_names = X_train_reduced . columnas ,
    class_names = [ "benigno" , "adware" , "malware" ],
    rounded = True ,
    filled = True
)

Fuente . from_file ( "android_malware.dot" )
```

Salida[...

[Centro de Git](#)

En []:



texto original
/ahora esta traducción
tu opinión servirá para ayudar a mejorar el Traductor de Google

